THESIS PHASE 2 · REFERENCE IMPLEMENTATION

Qwen PPO Reference Build

Reproducing the official Safe RLHF algorithm shape — reward model plus actor-critic PPO, no cost model, no Lagrangian — on Qwen + LoRA. This becomes the trusted baseline that GRPO and Minmax get measured against.

Workspace safe-rlhf/ **Base** PKU-Alignment upstream, pristine **Compute** mscluster · bigbatch → biggpu

Drafted 25 Aug 2026

What this stage is for

Phase 1 (GPT-2 + Detoxify, in `ppo_minmax_experiment/`) is closed. Its honest result: under matched KL, Minmax and PPO+KL tie — 2.0% vs 2.3% harm on one seed. The blocker was never the algorithm, it was the reward: Detoxify is a bounded, gameable detector, and a policy that learns to emit "Advertisements" scores perfectly on it while learning nothing about safety.

So Phase 2 changes the reward, not the mechanism. A learned preference model replaces the toxicity classifier, a real instruction-tuned LLM replaces GPT-2, and the framework becomes PKU's own rather than TRL. Before testing anything novel, we need this reference build to work — otherwise a later GRPO or Minmax result has nothing trustworthy to be compared against.

Carried forward from Phase 1

Matched KL across conditions or the comparison is confounded. Seed everything. Log entropy, KL, and clip fraction from step 0 — retrofitting them cost a whole rerun last time. Exclude empty responses from harm scoring.

Deliberately different this time

Reward comes from a trained preference model, so there is no single degenerate token that maxes it out. Detoxify survives only as a secondary eval metric, never as a training objective.

Target architecture

Four model roles. Only two of them carry trainable weights, and thanks to LoRA those weights are adapters measured in megabytes rather than full checkpoints.

ACTOR	**Qwen2.5-1.5B-Instruct + LoRA** — generates responses to PKU-SafeRLHF prompts. Only the adapters update. TRAINS

REFERENCE π_{REF}	**The same actor with adapters switched off** — supplies the KL anchor. Costs no extra memory because `disable_adapter()` reuses the base weights already resident. FREE
REWARD	**PKU-Alignment/beaver-7b-unified-reward** — scores each finished response. Frozen; never updated. LLaMA-family, so the trainer re-tokenizes between actor and RM automatically. FROZEN
CRITIC	**Qwen2ForScore on the Qwen base + LoRA** — per-token value estimates for the advantage. Starts with a randomly initialised value head. TRAINS

> **THE CRITIC CANNOT BE THE REWARD MODEL**
>
> Upstream feeds the critic the actor's raw `sequence` with no re-tokenization, and `rl_trainer.py` raises `ValueError` outright if the critic's tokenizer differs from the actor's. Since `--reward_critic_model_name_or_path` *defaults to the reward model path*, leaving it unset points a Qwen actor at a LLaMA critic and crashes on startup. It must be passed explicitly, pointing at a Qwen model.
>
> The reward model has no such constraint — `post_rollout()` calls `batch_retokenize()` whenever its tokenizer differs. That asymmetry is what makes the 7B Beaver RM usable here at all.

> **MEMORY BUDGET, ONE BIGGPU RTX 8000 (48 GB)**
>
> Actor ≈3 GB + critic ≈3 GB + frozen RM ≈14 GB ≈ **20 GB of weights** in bf16, with π_{ref} free. LoRA optimiser state is negligible. That leaves real headroom for activations at 512 tokens — and room to try the 3B actor later without re-planning.

Stages

Numbered because they are genuinely sequential — each one's exit gate is the next one's precondition. “Stage” here avoids collision with the thesis-level Phase 1 / Phase 2.

STAGE 0 Environment on the cluster — IN FLIGHT

Conda env from upstream's own `conda-recipe.yaml`, plus the two pins that upstream leaves dangerously loose.

- **peft** added — upstream lists it nowhere, and we need it.
- **mkl=2024.0.0** — later MKL drops the ittnotify symbols PyTorch links against.
- **transformers <4.47** — 5.x moved `tokenization_utils`, which safe-rlhf imports. Document this as a deliberate pin.

as a deliberate pin.

EXIT GATE

On a worker node: `torch.cuda.is_available()` is True, device count ≥ 1, and `from transformers.tokenization_utils import PaddingStrategy` succeeds.

STAGE 1 LoRA plumbing

NEXT

The only substantial engineering in this build. Upstream has zero LoRA support — `load_pretrained_models()` loads full weights straight into DeepSpeed. All local work; nothing blocked on the queue.

- Wrap the loaded model in `peft.get_peft_model()` behind new `--lora_*` flags.
- Thread those flags through the `finetune` and `algorithms/ppo` argument parsers.
- Route π_{ref} through `disable_adapter()` instead of loading a second full model — this is the memory win LoRA is meant to buy, so it belongs in the design, not bolted on later.
- Save and reload adapters rather than full checkpoints.

```
edits safe_rlhf/models/pretrained.py
edits safe_rlhf/trainers/rl_trainer.py
edits safe_rlhf/algorithms/ppo/main.py
```

EXIT GATE

A Qwen model loads with adapters attached, trainable parameters are a small percentage of total, and `disable_adapter()` reproduces the untouched base model's logits exactly.

STAGE 2 Prompt template and data path

QUEUED

Upstream hardcodes an Alpaca-style prompt in `configs/constants.py` (`BEGINNING OF CONVERSATION: USER: … ASSISTANT:`). Qwen2.5-Instruct was tuned on ChatML, so keeping the Alpaca framing imposes avoidable distribution shift right where LoRA has the least capacity to absorb it.

- Decide the template once and use it identically in training and evaluation.
- Confirm `PKU-SafeRLHF` prompt-only loading works unchanged — it is upstream's own dataset, so it should.

EXIT GATE

A batch of decoded prompts is printed and visually correct in the chosen template, with no doubled or missing special tokens.

STAGE 3 Smoke run on bigbatch

QUEUED

Prove the loop end to end on cheap hardware before touching the contested biggpu nodes — which is also what the cluster's own etiquette asks for. Small actor, small stand-in reward model, a handful of steps.

- Qwen2.5-0.5B actor and a small scoring model, purely to exercise the code path.
- Roughly 10 steps. The question is “does it run”, not “does it learn”.
- Verify the RM re-tokenization branch actually fires when tokenizers differ.

EXIT GATE

Ten steps complete without OOM or crash, rewards are finite and vary across samples, diagnostics land in the log, and a LoRA adapter is written to disk.

STAGE 4 Stage the real reward model

QUEUED

Download `beaver-7b-unified-reward` into `/datasets/wmaboa/` with `HF_HOME` pointed there, so the cache does not consume the 50 GB home quota.

- Sanity-check the score head: obviously harmful text should score clearly below obviously helpful text.
- A reward model that cannot separate those two is not worth spending biggpu hours on.

EXIT GATE

The RM loads through `AutoModelForScore` and produces a clearly separated score distribution on a hand-written probe set.

STAGE 5 Full reference run on biggpu

QUEUED

The actual baseline. Fixed seed, full diagnostics from step 0, checkpoint intervals set so a load-shedding event does not cost the whole run.

- Watch for the Phase 1 failure signature: entropy collapsing, KL going negative, clip fraction climbing.
- The critic head starts random — early value estimates are meaningless by construction, exactly as seen in Phase 1.

EXIT GATE

EXIT GATE

Training completes with entropy stable and KL positive throughout, and a saved adapter that generates coherent text.

STAGE 6 **Evaluation harness** QUEUED

Port the evaluation discipline from Phase 1 rather than rebuilding it — those rules were each learned from a specific bug.

- Empty responses excluded from harm scoring, never scored as the prompt.
- Report `harm_rate_nonempty` alongside `harm_rate_all`.
- Same decoding mode in eval as in training, or the comparison is meaningless.
- Baseline against untrained Qwen2.5-Instruct, the way raw GPT-2 anchored Phase 1.

EXIT GATE

A reproducible harm-rate number for the PPO reference build, with the untrained-Qwen anchor beside it — the figure GRPO and Minmax will be compared to.

Decisions taken, and why

DECISION	CALL	REASONING
SFT stage	Skip	PKU needed it because raw LLaMA-7B cannot follow instructions. Qwen2.5-Instruct already ships instruction-tuned — running SFT would re-teach what it knows and burn queue time. A deliberate scope decision, recorded as one.
Reward model	Use released Beaver 7B	PKU published trained reward models. Using theirs is more faithful to "official shape" than training our own, and removes an entire training stage.
Critic init	Qwen base + fresh head	Forced, not chosen: the trainer rejects any critic whose tokenizer differs from the actor's. Also far cheaper than a 7B critic.
PPO-Lag / cost model	Deferred	Deliberately out of this build. Plain PPO first; the Lagrangian variant is a later comparison, not a prerequisite.
Actor size	1.5B, then 3B	Memory budget leaves room for 3B, but proving the loop at 1.5B is faster and cheaper.

Where this could go wrong

DeepSpeed × PEFT friction

ZeRO-3 parameter partitioning and PEFT adapters are a historically awkward pairing. **Fallback:** drop to ZeRO-2 or plain DDP — with LoRA there is little full-weight optimiser state to shard, so ZeRO-3 is not buying much anyway.

Re-tokenization drift

`batch_retokenize` decodes and re-encodes with `skip_special_tokens=True`. Round-tripping Qwen text through a LLaMA tokenizer may not be lossless. **Fallback:** inspect decoded pairs at Stage 3; if scores look wrong, train a small Qwen reward model instead.

biggpu contention

Four of seven nodes were allocated and three down when last checked, with September–November historically full. **Fallback:** 1.5B actor and shorter runs fit bigbatch's 24 GB if the 7B RM is swapped for a smaller one.

Load shedding mid-run

The UPS survives roughly one event per day. **Fallback:** frequent `--save_interval` checkpoints and confirm resume works *before* the long run, not after losing one.

Explicitly out of scope

- × Retraining Beaver-7B, or any reward model, unless Stage 4's probe fails.
- × Implementing GRPO or the Minmax penalty in this codebase — that comes after the reference number exists.
- × PPO-Lag, cost models, and reward shaping.
- × Detoxify as a training objective. Secondary eval metric only.
- × Porting anything from `ppo_minmax_experiment/` into `safe-rlhf/`. Phase 1 stays where it is; this tree stays close to upstream so deviations remain legible.

Companion document: **mscluster Field Guide** — cluster hardware, job submission, and setup traps.

Every stage that produces a real result gets a Did / Found / Concluded entry in a worklog inside `safe-rlhf/`, in the same discipline as `ppo_minmax_experiment/worklog.md`. Dead ends count as entries.